

Vấn đáp Linux

Chuẩn bị trả lời câu hỏi của cô

La Minh Khánh – MSSV: 4004267

Bài 1 – Tạo / Xóa thư mục

1. Tệp bai1_LaMinhKhanh_4004267_Tao.sh

```
#!/bin/bash
TARGET_DIR="/home/minhkhánhla/test"
FOLDER_PATTERN="$TARGET_DIR/LaMinhKhanh_*"
FILE_TARGET="/home/minhkhánhla/laminhkhánhhk120252026/kiemtra0610/67
CS_1_LaMinhKhanh_4004267/bai1_LaMinhKhanh_4004267/ThongTin.txt"

if ls $FOLDER_PATTERN 1>/dev/null 2>&1
then
    echo "Phat hien thu muc ton tai."
    for i in $(seq 1 500)
    do
        rm -rf "$TARGET_DIR/LaMinhKhanh_$i"
    done
    echo "Da xoa xong."
else
    echo "Khong tim thay thu muc. Dang tao moi..."
    mkdir -p "$TARGET_DIR"
    for i in $(seq 1 500)
    do
        mkdir "$TARGET_DIR/LaMinhKhanh_$i"
    done

    for i in $(seq 1 500)
    do
        mv "$TARGET_DIR/LaMinhKhanh_$i" "$TARGET_DIR/SinhVienHUCE_$i"
    done

    for i in $(seq 1 500)
    do
        cp "$FILE_TARGET" "$TARGET_DIR/SinhVienHUCE_$i"
    done
fi
```

2. Tập bai1_LaMinhKhanh_4004267_Xoa.sh

```
#!/bin/bash
TARGET_DIR="/home/minhkhlanhla/test"
for i in $(seq 1 500)
do
    rm -rf "$TARGET_DIR/SinhVienHUCE_$i"
done
```

3. Tập ThôngTin.txt

```
Ho va ten: La Minh Khanh
MSSV: 4004267
Lop quan ly: 67CS1
Lop mon hoc: 67CS
De so: 1
```

Giải thích từng lệnh (để trả lời cô: “Bài này em có hướng làm như nào”)

Dưới đây em giải thích từng phần của hai script và file ThôngTin.txt — giải thích ngắn gọn, rõ ràng, cô hỏi chỗ nào thì em đọc thẳng phần tương ứng.

File: bai1_LaMinhKhanh_4004267_Tao.sh

```
#!/bin/bash
```

- **Shebang:** chỉ định shell sẽ dùng để chạy script (bash).

```
TARGET_DIR="/home/minhkhlanhla/test"
FOLDER_PATTERN="$TARGET_DIR/LaMinhKhanh_*"
FILE_TARGET="/home/minhkhlanhla/laminhkhlanhkh120252026/kiemtra0610/67
CS_1_LaMinhKhanh_4004267/bai1_LaMinhKhanh_4004267/ThôngTin.txt"
```

- **Khai báo biến:**

- TARGET_DIR: thư mục đích nơi tạo/xóa thư mục con.
- FOLDER_PATTERN: mẫu (glob) để kiểm tra xem có thư mục LaMinhKhanh_* tồn tại hay không.
- FILE_TARGET: đường dẫn tới file ThôngTin.txt sẽ được sao chép vào các thư mục sinh viên.

```
if ls $FOLDER_PATTERN 1>/dev/null 2>&1
then
    echo "Phat hien thu muc ton tai."
```

- **Kiểm tra tồn tại:** ls \$FOLDER_PATTERN thử liệt kê các file/thư mục khớp mẫu.

- `1>/dev/null 2>&1` chuyển cả stdout và stderr vào `/dev/null` — tức là lệnh chạy “âm thầm”, chỉ quan tâm đến mã thoát.
- Nếu `ls` thành công (có mục nào khớp) thì vào nhánh `then`.
- **Ghi chú:** dùng `ls` để kiểm tra tồn tại là phổ biến nhưng không phải cách an toàn nhất (xem phần cải thiện bên dưới).

```
for i in $(seq 1 500)
do
    rm -rf "$TARGET_DIR/LaMinhKhanh_$i"
done
echo "Da xoa xong."
```

- Nếu tìm thấy thư mục `LaMinhKhanh_*`: vòng `for` chạy `i` từ 1 đến 500 và `rm -rf` từng thư mục `LaMinhKhanh_i`.
 - `rm -rf` xóa đệ quy không hỏi — mạnh mẽ nhưng nguy hiểm nếu đường dẫn sai.
 - `"$TARGET_DIR/LaMinhKhanh_$i"`: trích dẫn để tránh lỗi khi có khoảng trắng trong tên.

```
else
    echo "Khong tim thay thu muc. Dang tao moi..."
    mkdir -p "$TARGET_DIR"
```

- Nếu không tìm thấy: vào nhánh `else`.
 - `mkdir -p`: tạo `TARGET_DIR` nếu chưa có; `-p` tạo luôn thư mục cha nếu cần và không báo lỗi nếu đã tồn tại.

```
for i in $(seq 1 500)
do
    mkdir "$TARGET_DIR/LaMinhKhanh_$i"
done
```

- Tạo 500 thư mục tên `LaMinhKhanh_1 ... LaMinhKhanh_500`.

```
for i in $(seq 1 500)
do
    mv "$TARGET_DIR/LaMinhKhanh_$i" "$TARGET_DIR/SinhVienHUCE_$i"
done
```

- Đổi tên (`rename`) từng thư mục `LaMinhKhanh_i` → `SinhVienHUCE_i`.
 - Lý do làm vậy: có thể mục đích là tạo tên chuẩn `SinhVienHUCE_` cho mỗi sinh viên; nhưng thực tế có thể tạo trực tiếp `SinhVienHUCE_*` luôn thay vì tạo rồi `mv`.

```
for i in $(seq 1 500)
do
    cp "$FILE_TARGET" "$TARGET_DIR/SinhVienHUCE_$i"
done
fi
```

- Sao chép file `ThôngTin.txt` vào mỗi thư mục `SinhVienHUCE_i`.

- cp "\$FILE_TARGET"\$TARGET_DIR/SinhVienHUCE_\$i": sao chép file (với cùng tên ThôngTin.txt) vào từng thư mục.

File: bai1_LaMinhKhanh_4004267_Xoa.sh

```
#!/bin/bash
TARGET_DIR="/home/minhkhanelhla/test"
for i in $(seq 1 500)
do
    rm -rf "$TARGET_DIR/SinhVienHUCE_$i"
done
```

- Script đơn giản: xóa 500 thư mục SinhVienHUCE_1 ... SinhVienHUCE_500.
- Dùng rm -rf nên cần cẩn trọng — chắc chắn TARGET_DIR đúng để tránh xóa nhầm.

File: ThôngTin.txt

```
Ho va ten: La Minh Khanh
MSSV: 4004267
Lop quan ly: 67CS1
Lop mon hoc: 67CS
De so: 1
```

- File thông tin mẫu; script sao chép file này vào mỗi thư mục sinh viên để mỗi thư mục có một file ThôngTin.txt chứa thông tin của sinh viên (ở đây là của em).

Những điểm cần chú ý / cải tiến (để nói thêm với cô)

1. Kiểm tra tồn tại thư mục:

- Thay vì if ls \$FOLDER_PATTERN ... em có thể dùng shopt -s nullglob rồi kiểm tra mảng, hoặc dùng compgen -G "\$FOLDER_PATTERN" hoặc if compgen -G "\$FOLDER_PATTERN" /dev/null; then ... — để tránh các lỗi khi glob không khớp.

2. Tránh tạo rồi mv:

- Thay vì tạo LaMinhKhanh_* rồi mv sang SinhVienHUCE_*, có thể tạo trực tiếp mkdir "\$TARGET_DIR/SinhVienHUCE_\$i".

3. An toàn khi xóa:

- rm -rf rất nguy hiểm. Có thể thêm kiểm tra -d trước khi xóa: if [-d "\$TARGET_DIR/SinhVienHUCE_\$i"]; then rm -rf ...; fi.

4. Tối ưu sao chép:

- Nếu muốn copy giữ nguyên quyền, timestamp có thể dùng cp -p hoặc install -m tùy mục đích.

5. Hiệu năng:

- Tạo 500 thư mục và copy file 500 lần là OK, nhưng nếu file lớn thì cân nhắc chỉ tạo symlink hoặc sao chép một lần nếu phù hợp.

6. Quyền (permission):

- Script phải được chạy bởi user có quyền ghi/xóa trong `TARGET_DIR` và đọc `FILE_TARGET`.

Gợi ý cách trả lời cô (tóm tắt 1–2 câu, trực tiếp)

"Em kiểm tra xem có thư mục mẫu `LaMinhKhanh_*` trong thư mục đích không. Nếu có thì script sẽ xóa các thư mục đó (dọn sạch). Nếu chưa có, script sẽ tạo 500 thư mục, đổi tên thành `SinhVienHUCE_1...SinhVienHUCE_500` và sao chép `ThongTin.txt` vào từng thư mục để mỗi thư mục có file thông tin mẫu. Em dùng vòng lặp `seq 1 500` để lặp cho đủ 500 thư mục và dùng `rm -rf, mkdir -p, mv, cp` để thực hiện các thao tác tệp."

Nếu cô muốn em giải thích cụ thể từng lệnh (từng dòng), em có thể đọc phần giải thích phía trên theo từng đoạn.

Nếu cô (hoặc em) muốn, em có thể:

- Viết lại script theo cách an toàn hơn (dùng `for i in {1..500}` / kiểm tra `-d` / tránh `ls`) — em sẽ gửi phiên bản sạch, ngắn gọn.

Tổng nói nhanh trước khi vào từng câu

Em chọn các lệnh theo tiêu chí: đơn giản, dễ hiểu cho mục tiêu bài (tạo/xóa nhiều thư mục, sao chép file mẫu), và chạy được trên bash mặc định. Nếu cô muốn em có thể trình bày phiên bản an toàn/hiệu quả hơn cho từng chỗ.

1) Câu: "Tại sao em dùng `if ls $FOLDER_PATTERN ...` để kiểm tra tồn tại mà không dùng `[ -d ... ]?`"

Trả lời mẫu: Mình dùng `ls $FOLDER_PATTERN` để kiểm tra xem có ít nhất một thư mục khớp mẫu `LaMinhKhanh_*` hay không (vì ta muốn biết có bất kỳ thư mục nào theo mẫu). `-d` kiểm tra một đường dẫn cụ thể có phải thư mục không, nên không hợp nếu mẫu có nhiều tên.
Khi nên đổi: Nếu chỉ cần kiểm tra một đường dẫn cố định thì dùng `if [ -d "$TARGET_DIR/LaMinhKhanh..." ]`.
 Nếu muốn cách an toàn hơn với glob, dùng `if compgen -G "$FOLDER_PATTERN" > /dev/null; then ...` hoặc bật `shopt -s nullglob` rồi kiểm tra mảng — cách này tránh lỗi khi glob không khớp.

2) Câu: "Tại sao em dùng `rm -rf` để xóa mà không dùng `rm -r` hoặc `rm -i`?"

Trả lời mẫu: `rm -rf` đảm bảo xóa đệ quy và **không hỏi** (force) nên phù hợp khi muốn dọn hàng loạt tự động trong script. `rm -r` sẽ hỏi khi gặp file chỉ đọc; `rm -i` sẽ hỏi tương tác nhiều lần — không phù hợp khi script chạy tự động.
Khi nên đổi: Nếu lo sợ xóa nhầm, dùng kiểm tra trước `if [ -d "$path" ]` rồi `rm -r` hoặc thêm prompt để xác nhận, hoặc di chuyển vào thùng rác thay vì xóa vĩnh viễn.

3) Câu: "Tại sao tạo LaMinhKhanh_* rồi mv sang SinhVienHUCE_* thay vì tạo thẳng SinhVienHUCE_*?"

Trả lời mẫu: Đây là cách em viết ban đầu (theo logic: tạo mẫu rồi đổi tên). Thực tế tạo thẳng SinhVienHUCE_\$i sẽ ngắn gọn hơn và tiết kiệm một vòng mv. Em viết như vậy do phát triển từng bước; nếu nộp chính thức em sẽ tạo thẳng.

Khi nên đổi: Nên đổi ngay để đơn giản: `mkdir "$TARGET_DIR/SinhVienHUCE_$i"`.

4) Câu: "Tại sao dùng cp để copy ThôngTin.txt vào từng thư mục thay vì ln (hardlink/symlink)?"

Trả lời mẫu: cp tạo bản sao riêng cho từng thư mục — nếu cần sửa file trong từng thư mục thì không ảnh hưởng file gốc. ln (hard link) hoặc ln -s (symbolic link) tiết kiệm không gian nhưng mọi thay đổi sẽ ảnh hưởng file gốc (hoặc link có thể hỏng nếu source bị xóa). Em chọn cp vì mục tiêu là mỗi thư mục có một file ThôngTin.txt độc lập.

Khi nên dùng ln: Nếu file lớn mà chỉ đọc và muốn tiết kiệm không gian/disk, có thể dùng link.

5) Câu: "Tại sao dùng seq 1 500 trong for mà không dùng for i in {1..500}?"

Trả lời mẫu: seq là tiện lợi và dễ đọc; {1..500} là brace expansion của bash (nhanh, không gọi tiến trình ngoài). Cách for i in {1..500} nhỏ gọn hơn và tránh gọi seq (một lệnh ngoài). Em có thể đổi sang {1..500} để script chạy nhanh hơn một chút.

Lưu ý tương thích: Brace expansion có sẵn trong bash; nếu muốn portable shell script POSIX có thể dùng seq.

6) Câu: "Tại sao em không quote biến khi dùng ls \$path -lR?"

Trả lời mẫu: Đây là sơ suất — biến nên được quote (ls -lR "\$path") để xử lý đường dẫn có khoảng trắng hoặc ký tự đặc biệt. Ở môi trường lab đường dẫn không có khoảng trắng nên vẫn chạy, nhưng chuẩn là luôn quote biến. Em sẽ sửa khi nộp chính thức.

7) Câu: "Tại sao em dùng ls -lR | grep ^d | wc -l để đếm folder thay vì find?"

Trả lời mẫu: ls -lR | grep là cách nhanh để lấy ý tưởng và dễ viết với kiến thức cơ bản. Tuy nhiên find "\$path" -type d | wc -l chính xác và an toàn hơn (không bị ảnh hưởng bởi tên chứa newline / ký tự lạ). Nếu cần độ chính xác, em sẽ dùng find.

8) Câu: "Tại sao không kiểm tra quyền (permission) trước khi cp hoặc mv?"

Trả lời mẫu: Script hiện giả định người chạy có quyền cần thiết. Để an toàn hơn nên kiểm tra if [-r "\$FILE_TARGET"] trước khi cp và if [-w "\$path"] trước khi mv hoặc xử lý lỗi sau khi lệnh chạy (kiểm tra \$?). Em có thể thêm các kiểm tra này.

9) Câu: "Tại sao dùng exit 1 khi thoát (trong script khác) thay vì exit 0?"

Trả lời mẫu: exit 1 biểu thị script thoát với mã lỗi (không thành công). Trong một số kịch bản em dùng exit 1 để biểu thị rằng hành động thoát là do điều kiện không như mong muốn. Nếu thoát bình thường thì nên dùng exit 0. Em sẽ điều chỉnh cho hợp ngữ nghĩa.

10) Câu: "Tại sao không dùng `set -euo pipefail` ở đầu script để an toàn hơn?"

Trả lời mẫu: `set -euo pipefail` giúp script fail sớm khi có lỗi, bắt biến chưa khởi tạo và lỗi trong pipeline — rất hữu ích để tránh lỗi im lặng. Em không thêm ban đầu để kịch bản đơn giản cho ví dụ, nhưng trong thực tế, nhất là khi thao tác xóa file, em sẽ bật options này để an toàn.

11) Câu: "Tại sao không dùng `printf` thay cho `echo` trong một số chỗ?"

Trả lời mẫu: `printf` cho kiểm soát định dạng tốt hơn và hành vi ổn định (ví dụ `echo -n` khác nhau giữa shell). `echo` đơn giản và dễ đọc nên em dùng cho output thông báo. Nếu cần in chính xác khoảng trắng/format, em sẽ dùng `printf`.

12) Câu: "Nếu đường dẫn mục tiêu sai, sao bảo đảm script không xóa nhầm toàn bộ ổ đĩa?"

Trả lời mẫu: Để an toàn, cần kiểm tra biến `TARGET_DIR` trước khi dùng `rm -rf`, ví dụ `if [ -z "$TARGET_DIR" ] || [ "$TARGET_DIR" = "/" ]; then echo "Unsafe target"; exit 1; fi`. Ngoài ra có thể in debug (`echo` các lệnh) hoặc chạy ở chế độ “đóng băng” (dry-run) trước khi xóa thực.

13) Câu: "Tại sao không dùng parallelization hoặc `xargs -P` để nhanh hơn khi tạo/xóa 500 thư mục?"

Trả lời mẫu: Tạo/xóa 500 thư mục trên hầu hết hệ thống là nhanh và đơn giản bằng vòng lặp tuần tự; thêm song song làm script phức tạp hơn và có thể gây tải I/O. Nếu làm trên cluster hoặc file system chậm thì có thể cân nhắc `xargs -P` hoặc jobs background, nhưng cần cẩn thận race condition và limit tài nguyên.

14) Câu: "Thay vì sao chép 500 lần, có cách nào chỉ copy một lần?"

Trả lời mẫu: Nếu file chỉ đọc và không cần mỗi thư mục một bản độc lập, có thể copy một lần vào một thư mục mẫu rồi dùng `cp -al` (hardlink clone) hoặc tạo symlink vào file mẫu để tiết kiệm không gian. Nhưng với bài yêu cầu mỗi sinh viên có file `ThôngTin.txt` riêng (để sửa từng file), cần `cp`.

Kết luận — gợi ý nói với cô

“Em chọn các lệnh vì đơn giản, trực quan và phù hợp với mục tiêu bài: tự động tạo 500 thư mục và đặt file mẫu vào từng thư mục. Nếu cô muốn em tối ưu — ví dụ dùng `find` để đếm, quote mọi biến, dùng brace expansion `{1..500}`, thêm kiểm tra quyền và `set -euo pipefail` — em sẽ chỉnh lại ngay.”

Nếu cô hay hỏi thêm chỗ nào cụ thể (ví dụ: an toàn khi `rm -rf`, đếm file chính xác, hay tối ưu performance), em nói tên mục đó — em sẽ đưa câu trả lời chi tiết + ví dụ lệnh thay thế luôn. Muốn em viết phiên bản script **đã sửa** theo các best-practices không?

Bài 2 – Tìm tham số ngắn nhất

bai2_LaMinhKhanh_4004267.sh

```
#!/bin/bash

if [ $# -lt 1 ];
then
    echo "Khong co tham so nao"
    exit 1
fi

minLength=$(expr length $1)
minIndex=1;
minValue=$1;

count=0

for i in $@
do
    count=$((count+1))
    tempLength=$(expr length $i)
    if [ $tempLength -lt $minLength ];
    then
        minValue=$i
        minLength=$tempLength
        minIndex=$count
    fi
done

echo
echo "Tham so ngan nhat dau tien la: $minValue"
echo "So ky tu la: $minLength"
echo "Vi tri la: $minIndex"
```

Giải thích từng lệnh — cách em trình bày khi cô hỏi "Bài này em có hướng làm như nào"

Dưới đây em giải thích từng phần của script bai2_LaMinhKhanh_4004267.sh (dòng-theo-dòng) — cô hỏi chỗ nào thì em đọc thẳng phần tương ứng.

#!/bin/bash

- Shebang: cho biết script sẽ chạy bằng bash.

```
if [ $# -lt 1 ];
then
    echo "Khong co tham so nao"
    exit 1
fi
```

- "\$#" là số lượng tham số dòng lệnh.
- if [\$# -lt 1] kiểm tra xem có ít hơn 1 tham số (tức là không có tham số) hay không.

- Nếu không có tham số: in ra "Khong co tham so nao" và `exit 1` (kết thúc script với mã lỗi 1).

```
minLength=$(expr length $1)
minIndex=1;
minValue=$1;
```

- `minLength=$(expr length $1)` dùng `expr length` để lấy độ dài (số ký tự) của tham số đầu tiên (`$1`) và gán cho `minLength`.
- `minIndex=1` khởi tạo vị trí của tham số ngắn nhất hiện tại là 1 (bắt đầu từ tham số đầu).
- `minValue=$1` lưu giá trị tham số đầu làm giá trị ngắn nhất tạm thời.
- **Ghi chú:** dùng `expr` hoạt động nhưng cần cẩn thận với các tham số chứa khoảng trắng hoặc ký tự đặc biệt.

```
count=0
```

- Khởi tạo bộ đếm `count` để biết vị trí (stt) của tham số hiện tại khi lặp.

```
for i in $@
do
    count=$((count+1))
    tempLength=$(expr length $i)
    if [ $tempLength -lt $minLength ];
    then
        minValue=$i
        minLength=$tempLength
        minIndex=$count
    fi
done
```

- `for i in $@` lặp qua **tất cả** tham số dòng lệnh. (`$@` chứa tất cả tham số)
 - **Lưu ý an toàn:** `for i in $@` (không có ngoặc kép) sẽ tách tham số theo whitespace, vì vậy tham số có khoảng trắng bị tách thành nhiều phần. Nên dùng `for i in "$@"` để bảo toàn tham số có khoảng trắng.
- Trong mỗi vòng:
 - `count=$((count+1))` tăng bộ đếm vị trí.
 - `tempLength=$(expr length $i)` lấy độ dài của tham số hiện tại và gán vào `tempLength`.
 - `if [ $tempLength -lt $minLength ]; then ... fi` so sánh số nguyên: nếu tham số hiện tại ngắn hơn `minLength` thì cập nhật `minValue`, `minLength`, `minIndex` (lưu giá trị, độ dài và vị trí mới).
- Kết quả: sau vòng lặp, `minValue` chứa **tham số ngắn nhất đầu tiên** gặp được, `minLength` là độ dài của nó, `minIndex` là vị trí xuất hiện trong danh sách tham số.

```
echo
echo "Tham so ngan nhat dau tien la: $minValue"
echo "So ky tu la: $minLength"
echo "Vi tri la: $minIndex"
```

- In ra kết quả: tên tham số ngắn nhất, số ký tự, và vị trí (vị trí bắt đầu từ 1).

Cách em tóm tắt để trả lời cô (1–2 câu)

"Em kiểm tra xem script có tham số truyền vào không; nếu không có thì báo lỗi. Em lặp qua toàn bộ tham số, đo độ dài từng tham số, và giữ lại tham số có độ dài nhỏ nhất (nếu có nhiều tham số bằng độ dài nhỏ nhất thì script trả tham số **đầu tiên** xuất hiện). Cuối cùng in ra giá trị ngắn nhất, số ký tự và vị trí."

Một vài lưu ý & gợi ý cải tiến (để nói thêm với cô nếu cần)

1. Bảo toàn tham số có dấu cách:

- Thay `for i in $@` bằng `for i in "$@"` để tham số chứa khoảng trắng không bị cắt.

2. Dùng cú pháp Bash gọn và an toàn hơn:

- Thay `expr length` bằng `${#var}` để lấy chiều dài: `minLength=${#1}` và `tempLength=${#i}`. Cú pháp này nhanh hơn, không cần gọi chương trình ngoài.

3. Đếm ký tự đa byte (UTF-8):

- `${#var}` đếm số ký tự theo cách của shell và có thể phụ thuộc locale; nếu muốn chắc chắn cho UTF-8, dùng: `tempLength=$(printf '%s' "$i" | wc -m)` (đếm ký tự).

4. Quoting mọi biến khi dùng:

- Khi in/só sánh nên quote biến: `if [ "$tempLength" -lt "$minLength" ]` để tránh lỗi khi biến rỗng hoặc chứa ký tự lạ.

5. Xử lý trường hợp tham số rỗng:

- Nếu người dùng truyền "" (tham số rỗng), `minLength` có thể = 0 — script hoạt động OK nhưng cần lưu ý nếu muốn loại trừ tham số rỗng.

6. Thông báo lỗi ra `stderr`:

- In thông báo lỗi sang `stderr`: `echo "Khong co tham so nao" >&2`.
-

Phiên bản script đã được chỉnh cho an toàn hơn (em có thể dùng để nộp nếu cô cần)

```
#!/bin/bash

if [ $# -lt 1 ]; then
    echo "Khong co tham so nao" >&2
    exit 1
fi

minValue="$1"
minLength=${#1}
```

```

minIndex=1
count=0

for arg in "$@"; do
    count=$((count+1))
    tempLength=${#arg}      # hoặc: tempLength=$(printf '%s' "$arg" | wc -m) cho UTF-8
    if [ "$tempLength" -lt "$minLength" ]; then
        minValue="$arg"
        minLength=$tempLength
        minIndex=$count
    fi
done

echo
echo "Tham số ngắn nhất đầu tiên là: $minValue"
echo "Số ký tự là: $minLength"
echo "Vị trí là: $minIndex"

```

Nếu cô muốn, em có thể:

- Giải thích chi tiết từng dòng theo thứ tự khi thuyết trình; hoặc
- Gửi phiên bản script có thêm `--help` và ví dụ chạy; hoặc
- Chạy thử vài ví dụ mẫu và cho kết quả mẫu để minh họa (ví dụ: `./script.sh "apple" "a b" "xy"` -> ...).

Cần cái nào em làm luôn cho cô ạ?

Dưới đây là **một loạt câu hỏi cô có thể hỏi** kiểu “Tại sao em dùng lệnh này mà không dùng lệnh khác?” kèm **các câu trả lời mẫu** ngắn, rõ ràng và **gợi ý thay thế** — em đọc thẳng phần “Trả lời mẫu”, còn phần “Khi nên đổi / gợi ý” để trả lời sâu hơn nếu cô truy vấn kỹ.

Tổng tóm tắt (nói ngay)

Em dùng `expr length` + vòng `for` để đơn giản hóa: lặp qua từng tham số, đo độ dài, và cập nhật tham số ngắn nhất. Nếu cô muốn phiên bản an toàn/hay hiệu quả hơn em có thể trình bày ngay.

1) “Tại sao em dùng `if [ $# -lt 1 ]` để kiểm tra tham số mà không dùng `if [ -z "$1" ]`?”

Trả lời mẫu:

`[ $# -lt 1 ]` kiểm tra **số lượng tham số** truyền vào (rõ ràng và trực tiếp). `-z "$1"` kiểm tra `"$1"` có rỗng hay không — hai cách tương đương trong trường hợp này nhưng dùng `$#` thể hiện rõ ý “không có tham số nào”.

Khi đổi:

Dùng `if [ -z "$1" ]` cũng ổn. Nếu cần kiểm tra số tham số tối thiểu (ví dụ `>= 2`) thì `$#` là lựa chọn tự nhiên.

2) “Tại sao em dùng `expr length $1` thay `${#1}`?”

Trả lời mẫu:

`expr` là cách cổ điển và portable; mình đã quen dùng `expr length`.

Khi nên đổi:

Trong `bash`, `${#var}` là nhanh hơn và không tạo tiến trình con. Thay luôn bằng `minLength=${#1}` để code ngắn, an toàn và hiệu quả hơn.

3) “Tại sao `for i in $@` mà không dùng `for i in "$@"`?”

Trả lời mẫu:

`for i in $@` tách tham số theo whitespace, còn `for i in "$@"` bảo toàn từng tham số nguyên vẹn (đúng ý là lặp theo tham số). Mình viết tắt nhưng đúng là nên dùng `"$@"`.

Gợi ý:

Luôn dùng `for i in "$@"` để xử lý tham số có khoảng trắng.

4) “Tại sao em dùng `count` và tăng thủ công thay vì dùng chỉ số mảng hoặc `for ((i=1;i<=...))`?”

Trả lời mẫu:

Dùng `count` đơn giản, rõ ràng, dễ đọc khi lặp qua `"$@"`. Nếu muốn dùng chỉ số, có thể chuyển các tham số vào mảng rồi lặp theo chỉ số.

Khi đổi: Dùng:

```
count=0
for arg in "$@"; do
    count=$((count+1))
    ...
done
```

hoặc:

```
args=("$@")
for i in "${!args[@]}"; do
    # i là index từ 0
done
```

5) “Tại sao không quote biến trong `expr length $i` hay khi so sánh `if [ $tempLength -lt $minLength ]`?”

Trả lời mẫu:

Đây là sơ suất. Biến nên được quote khi có khả năng rỗng hoặc chứa ký tự lạ.

Gợi ý: Viết an toàn hơn:

```
tempLength=${#i}
if [ "$tempLength" -lt "$minLength" ]; then ...
```

6) “Tại sao em không xử lý tham số rỗng ("") đúng cách?”

Trả lời mẫu:

Script hiện chấp nhận tham số rỗng như một tham số có độ dài 0; nếu muốn bỏ qua tham số rỗng thì cần kiểm tra `if [ -z "$i" ] ; then continue; fi`.

Khi đổi:

Nếu không muốn tính tham số rỗng thì thêm `if [ -z "$i" ] ; then continue; fi` trong vòng lặp.

7) “Tại sao dùng `exit 1` khi không có tham số thay vì in ra usage rồi `exit 0`?”

Trả lời mẫu:

`exit 1` báo lỗi (không có tham số) — hợp lý vì script không hoàn thành mục tiêu. Nếu muốn nhẹ nhàng hơn có thể in hướng dẫn và `exit 0`, nhưng theo chuẩn, thiếu tham số là trường hợp lỗi nên `exit 1` là phù hợp.

8) “Làm sao đếm ký tự Unicode (UTF-8) đúng?”

Trả lời mẫu:

`${#var}` đếm ký tự theo shell và thường đủ. Để chắc chắn cho UTF-8, có thể dùng:

```
tempLength=$(printf '%s' "$i" | wc -m)
```

`wc -m` đếm ký (multi-byte safe). Dùng khi tham số có ký tự Unicode.

9) “Tại sao không in ví dụ chạy hoặc thêm `-help`?”

Trả lời mẫu:

Script simple phục vụ bài tập; nhưng thêm `--help/usage` là hay để người dùng biết cú pháp. Em có thể bổ sung nhanh:

```
if [ "$1" = "--help" ]; then
    echo "Usage: $0 arg1 arg2 ..."
    exit 0
fi
```

10) “Tại sao không dùng `awk/python` để xử lý chuỗi đúng hơn?”

Trả lời mẫu:

Mục tiêu bài là luyện bash cơ bản, nên dùng công cụ shell. Nếu cần xử lý phức tạp về encoding hoặc logic, chuyển sang `awk/python` sẽ mạnh mẽ và ngắn gọn hơn.

11) “Tại sao em không kiểm tra lỗi/exception (ví dụ tham số không phải chuỗi)?”

Trả lời mẫu:

Ở đây tham số chấp nhận bất kỳ chuỗi nào. Nếu cần giới hạn (ví dụ chỉ letters) có thể kiểm tra bằng regex: `if [ [ ! "$i" =~ ^[[:alpha:]]+$ ] ]; then ... fi.`

12) “Có cách nào trả nhiều kết quả (nhiều tham số cùng độ dài ngắn nhất) không?”

Trả lời mẫu:

Script hiện chỉ lưu **tham số ngắn nhất đầu tiên**. Nếu muốn liệt kê tất cả kết quả có cùng độ dài nhỏ nhất, ta có thể chạy hai lần: lần 1 tìm `minLength`, lần 2 in tất cả tham số có độ dài `== minLength`, hoặc lưu danh sách khi cập nhật bằng mảng.

Phiên bản ngắn (an toàn hơn) — em có thể nộp thay thế ngay

```
#!/bin/bash
if [ $# -lt 1 ]; then
    echo "Khong co tham so nao" >&2
    exit 1
fi

minValue="$1"
minLength=${#1}
minIndex=1
count=0

for arg in "$@"; do
    count=$((count+1))
    tempLength=${#arg} # hoặc: tempLength=$(printf '%s' "$arg" | wc -m) cho UTF-8
    if [ "$tempLength" -lt "$minLength" ]; then
        minValue="$arg"
        minLength=$tempLength
        minIndex=$count
    fi
done

echo
echo "Tham so ngan nhat dau tien la: $minValue"
echo "So ky tu la: $minLength"
echo "Vi tri la: $minIndex"
```

Giải thích ngắn: dùng `${#var}`, dùng `for arg in "$@"` và quote mọi biến — an toàn hơn, nhanh hơn.

Nếu cô hỏi chi tiết một dòng cụ thể, em có thể đọc nguyên văn phần “Trả lời mẫu” tương ứng. Muốn em chuẩn hóa **toàn bộ script** theo best-practices (thêm `--help`, UTF-8 safe, liệt kê tất cả tham số cùng độ dài nhỏ nhất, hoặc test ví dụ minh họa), nói em sẽ chỉnh luôn.

Bài 3 – Menu tương tác (Count / Process / Move / Exit)

bai3_LaMinhKhanh_4004267.sh

```
#!/bin/bash

while true
do
    echo
    echo "1. Count"
    echo "2. Process"
    echo "3. Move"
    echo "4. Exit"
    read -p "Xin moi chon: " input
    case $input in
        1|c|C)
            path="./"
            read -p "Nhap vao 1 duong dan (mac dinh: './') : " path
            echo "So luong folder la: $(ls $path -lR | grep ^d | wc -l)"
            echo "So luong files: $(ls $path -lR | grep ^- | wc -l)"
            ;;
        2|p|P) echo
            echo "Cac tien trinh dang chay: "
            echo "$ps -a"
            ;;
        3|m|M) echo
            while true
            do
                read -p "Nhap vao ten file: " file
                if [ -f "$file" ];
                then
                    break
                else
                    echo "File KHONG ton tai"
                fi
            done

            while true
            do
                read -p "Nhap vao duong dan: " path
                if [ -d "$path" ];
                then
                    break
                else
                    echo "Duong dan KHONG ton tai"
                fi
            done
            mv "$file" "$path"
            echo "=>> Di chuyen thanh cong"
            ;;
        4|e|E)
            echo "Goodbye"
    esac
done
```

```

        exit 1
    ;;
esac
done

```

Giải thích từng lệnh — cách em trình bày khi cô hỏi "Bài này em có hướng làm như nào"

Dưới đây em giải thích theo thứ tự xuất hiện trong script `bai3_LaMinhKhanh_4004267.sh`. Em có thể đọc thẳng từng phần khi thuyết trình cho cô.

```
#!/bin/bash
```

- **Shebang:** cho biết script sẽ được thực thi bằng `bash`.

```

while true
do
    ...
done

```

- **Vòng lặp vô hạn:** hiển thị menu liên tục cho đến khi người dùng chọn thoát. `true` luôn trả về thành công nên vòng lặp không dừng trừ khi gặp `break` hoặc `exit`.

```

echo
echo "1. Count"
echo "2. Process"
echo "3. Move"
echo "4. Exit"

```

- **In menu:** `echo` in ra dòng trống (để tạo khoảng cách) và các dòng chữ mô tả 4 lựa chọn của người dùng.

```
read -p "Xin moi chon: " input
```

- **Đọc lựa chọn của người dùng:** `read -p` hiện prompt rồi lưu giá trị nhập vào biến `input`.

```

case $input in
    1|c|C)
        ...
        ;;
    2|p|P)
        ...
        ;;
    3|m|M)
        ...
        ;;
    4|e|E)
        ...
        ;;
esac

```


- **Xử lý lựa chọn:** case so sánh input với các mẫu. Ví dụ 1|c|C) nghĩa là nếu người dùng nhập 1 hoặc c hoặc C thì chạy nhánh này. Mỗi nhánh kết thúc bằng ;;.

Nhánh 1 — Count (đếm folder và file)

```
path="./"
read -p "Nhập vào 1 đường dẫn (mặc định: './') : " path
echo "Số lượng folder là: $(ls $path -lR | grep ^d | wc -l)"
echo "Số lượng files: $(ls $path -lR | grep ^- | wc -l)"
```

- path="./": khởi tạo biến path mặc định là thư mục hiện tại.
- read -p ... path: hỏi người dùng nhập đường dẫn; nếu họ nhập rỗng thì path vẫn là "./" do đã khởi tạo trước.
- ls \$path -lR: liệt kê đệ quy tất cả file/thư mục trong path ở dạng chi tiết (-lR).
 - **Lưu ý:** \$path chưa được đặt trong dấu ngoặc kép — sẽ gây lỗi nếu đường dẫn có khoảng trắng. Nên dùng ls "\$path" -lR.
- grep ^d: lọc các dòng bắt đầu bằng d (dòng của thư mục trong output ls -l).
- wc -l: đếm số dòng — kết quả là số thư mục.
- Tương tự grep ^- lọc các file thông thường (dấu - ở đầu dòng ls -l) để đếm file.
- **Hạn chế/ghi chú:** Dùng ls + grep để đếm có thể sai/không an toàn với tên file đặc biệt; find là công cụ chính xác hơn:
 - Đếm thư mục: find "\$path" -type d | wc -l
 - Đếm file: find "\$path" -type f | wc -l

Nhánh 2 — Process (liệt kê tiến trình)

```
2|p|P) echo
echo "Các tiến trình đang chạy: "
echo "$(ps -a)"
;;
```

- ps -a: in danh sách các tiến trình của các terminal (các tiến trình có controlling terminal).
- echo "\$(ps -a)" bọc kết quả ps -a vào echo — hiệu quả tương tự in trực tiếp ps -a. Có thể đơn giản dùng ps -a (không cần echo), hoặc ps aux để xem đầy đủ hơn (tất cả tiến trình).

Nhánh 3 — Move (di chuyển file)

```
3|m|M) echo
while true
do
    read -p "Nhập vào tên file: " file
```

```

if [ -f "$file" ];
then
    break
else
    echo "File KHÔNG tồn tại"
fi
done

```

- Vòng `while true` hỏi tên file cần di chuyển cho đến khi user nhập một file **tồn tại**.
- `-f "$file"` kiểm tra file bình thường tồn tại hay không. Dấu ngoặc kép `"$file"` đã dùng đúng để hỗ trợ tên có khoảng trắng.

```

while true
do
    read -p "Nhập vào đường dẫn: " path
    if [ -d "$path" ];
    then
        break
    else
        echo "Đường dẫn KHÔNG tồn tại"
    fi
done

```

- Vòng `while` thứ hai hỏi đường đích (`path`) cho đến khi user nhập một **thư mục tồn tại**.
- `-d "$path"` kiểm tra có phải là thư mục hay không.

```

mv "$file" "$path"
echo "=>> Di chuyển thành công"

```

- `mv "$file" "$path"` di chuyển file vào thư mục đích. Dấu ngoặc kép bảo đảm tên file/đường dẫn có khoảng trắng vẫn xử lý đúng.
- In thông báo thành công.

Nhánh 4 — Exit (thoát)

```

4|e|E)
    echo "Goodbye"
    exit 1
;;

```

- In "Goodbye" và `exit 1` thoát chương trình.
- **Ghi chú:** `exit 1` thông thường biểu thị thoát với mã lỗi; nếu muốn thoát thành công thì dùng `exit 0`. Dùng `exit 1` không sai nhưng ý nghĩa là “thoát lỗi”.

Một số lưu ý & gợi ý cải tiến (để nói thêm với cô)

1. Quoting khi dùng biến trong lệnh:

- Thay `ls $path -lR` bằng `ls -lR "$path"` (hoặc tốt hơn dùng `find "$path" ...`) để tránh lỗi với đường dẫn có khoảng trắng hoặc ký tự đặc biệt.
2. **Dùng `find` thay vì `ls | grep` để đếm file/folder** — `find` cho kết quả chính xác hơn và an toàn với tên file lạ:
 - `find "$path" -type d | wc -l` (đếm thư mục)
 - `find "$path" -type f | wc -l` (đếm file)
 3. **Đọc nhập an toàn:** nên dùng `read -r` để tránh xử lý escape sequences: `read -r -p "... " input`.
 4. **In tiến trình:** `ps -a` có thể không hiển thị tất cả tiến trình; nếu cần đầy đủ, dùng `ps aux` hoặc `ps -ef`.
 5. **Mã thoát:** dùng `exit 0` khi thoát bình thường thay vì `exit 1`.
 6. **Thông báo lỗi rõ ràng:** khi `mv` thất bại (ví dụ không có quyền), có thể kiểm tra mã thoát và thông báo lỗi:

```
if mv "$file" "$path"; then
    echo "Di chuyển thành công"
else
    echo "Di chuyển thất bại" >&2
fi
```

7. **Tính tiện lợi UI:** có thể thêm `clear` trước khi in menu để giao diện sạch hơn, hoặc thêm lựa chọn `help`.

Cách em tóm tắt để trả lời cô (1–2 câu)

"Script hiện một menu vòng lặp cho phép: (1) đếm số thư mục và file trong một đường dẫn, (2) hiển thị các tiến trình đang chạy, (3) di chuyển một file vào thư mục khác (có kiểm tra tồn tại trước), và (4) thoát. Em dùng `read` để nhận lựa chọn/đầu vào, `case` để xử lý từng lựa chọn, `ls/grep/wc` (có thể thay bằng `find`) để đếm, `ps` để liệt kê tiến trình, và `mv` để di chuyển file."

Nếu cô muốn, em có thể gửi phiên bản script **đã chỉnh sửa an toàn** (dùng `find`, `read -r`, `exit 0`, kiểm tra lỗi `mv`, etc.) để nộp luôn. Muốn em gửi luôn không ạ?

Dưới đây là **một bộ mô phỏng các câu hỏi cô có thể hỏi** kiểu "Tại sao em dùng lệnh này mà không dùng lệnh khác?" kèm **câu trả lời mẫu** ngắn, rõ ràng + **gợi ý khi nên đổi**. Em có thể đọc thẳng phần "**Trả lời mẫu**" khi trình bày; phần "**Gợi ý thay thế / khi đổi**" để trả lời sâu hơn nếu cô truy vấn kỹ.

1) "Tại sao em dùng `while true` để hiển thị menu thay vì một cấu trúc khác?"

Trả lời mẫu:

`while true` tạo vòng lặp vô hạn cho menu, phù hợp khi muốn menu lặp liên tục cho tới khi người dùng chọn thoát. Cách này đơn giản, dễ hiểu cho mục đích bài tập.

Khi đổi / gợi ý:

Có thể dùng `until false` (tương đương) hoặc dùng `select` của bash để hiện menu tự động có số lựa chọn — `select` giúp người dùng chọn bằng số và tự in menu, tiện cho UI.

2) “Tại sao em dùng `read -p` mà không dùng `read -r -p`?”

Trả lời mẫu:

`read -p` in prompt và đọc input; trong kịch bản đơn giản này đủ dùng.

Khi đổi / gợi ý:

Nên dùng `read -r -p` vì `-r` ngăn backslash được xử lý như escape (an toàn hơn với input chứa `\`). Thói quen tốt là luôn dùng `read -r`.

3) “Tại sao em dùng `case $input in 1|c|C)` thay vì chỉ `1)`?”

Trả lời mẫu:

Mình cho phép người dùng nhập nhiều kiểu hợp lệ (số 1 hoặc chữ `c/C`) để UI linh hoạt hơn — người dùng có thể gõ phím tắt chữ.

Khi đổi / gợi ý:

Nếu muốn bắt chính xác 1..4 thì chỉ để `1)...`4). Nếu muốn phân biệt chữ hoa/thường, vẫn dùng `|` để liệt kê nhiều mẫu.

4) “Tại sao em dùng `ls $path -lR | grep ■d | wc -l` để đếm folder thay vì `find`?”

Trả lời mẫu:

`ls -lR | grep` là cách quen và nhanh để viết khi học, cho kết quả nhìn được.

Khi đổi / gợi ý:

`ls` có vấn đề với tên file có newline, khoảng trắng và một số ký tự đặc biệt.

`find "$path" -type d | wc -l` an toàn và chính xác hơn (đề xuất dùng `find` thay cho `ls` trong script thật sự). Ví dụ:

```
dir_count=$(find "$path" -type d | wc -l)
file_count=$(find "$path" -type f | wc -l)
```

5) “Tại sao dùng `ps -a` và `echo "$(ps -a)"` thay vì `ps aux` hay chỉ `ps -a`?”

Trả lời mẫu:

`ps -a` liệt kê tiến trình liên quan tới terminals, đủ cho minh họa. `echo "$(ps -a)"` in kết quả ra màn hình (có thể in trực tiếp `ps -a`).

Khi đổi / gợi ý:

Nếu cần danh sách đầy đủ (tất cả tiến trình), dùng `ps aux` hoặc `ps -ef`. Không cần bọc `ps` bằng `echo "$( ... )"` — chỉ gõ `ps -a` hoặc `ps aux` là rõ ràng hơn.

6) “Tại sao em kiểm tra file với -f và thư mục với -d?”

Trả lời mẫu:

-f kiểm tra file thường tồn tại; -d kiểm tra thư mục tồn tại — đây là cách chuẩn để đảm bảo nhập hợp lệ trước khi mv.

Gợi ý:

Đúng là nên dùng. Có thể bổ sung kiểm tra quyền -r/-w (đọc/ghi) trước khi thao tác để báo lỗi rõ ràng hơn.

7) “Tại sao em dùng mv "\$file"\$path" thay vì cp hoặc rsync?”

Trả lời mẫu:

Nhiệm vụ là **di chuyển** file, nên dùng mv đơn giản và nhanh (không tạo bản sao). cp tạo bản sao, rsync mạnh hơn nếu cần đồng bộ hoặc giữ quyền/thuộc tính.

Khi đổi:

Nếu muốn giữ lại bản gốc, dùng cp. Nếu cần di chuyển qua mạng hoặc giữ permission/time, dùng rsync -a.

8) “Tại sao em không quote \$path khi dùng ls \$path -lR?” (nếu cô để ý)

Trả lời mẫu:

Đây là sơ suất; biến nên luôn được quote để xử lý đường dẫn có khoảng trắng: ls -lR "\$path" hoặc dùng find "\$path" Em sẽ sửa cho an toàn.

Gợi ý:

Luôn quote biến khi dùng trong lệnh: "\$var".

9) “Tại sao em dùng exit 1 khi thoát chương trình thay vì exit 0?”

Trả lời mẫu:

exit 1 báo thoát với mã lỗi (không thành công). Trong trường hợp chọn Exit từ menu, hợp lý hơn là exit 0 để biểu thị kết thúc bình thường. Em có thể đổi thành exit 0 để rõ nghĩa.

Gợi ý:

Dùng exit 0 cho thoát thành công, exit khác cho lỗi.

10) “Tại sao em dùng echo để thông báo success thay vì kiểm tra kết quả lệnh mv rồi thông báo?”

Trả lời mẫu:

Mình in thông báo sau mv theo giả định mv thành công.

Khi đổi / gợi ý:

Nên kiểm tra mã thoát của mv để chắc chắn:

```
if mv "$file" "$path"; then
    echo "Di chuyển thành công"
else
    echo "Di chuyển thất bại" >&2
```

```
fi
```

11) “Tại sao không thêm `clear` hoặc in menu đẹp hơn mỗi vòng lặp?”

Trả lời mẫu:

Script đơn giản nên để nguyên; nhưng thêm `clear` trước khi in menu giúp giao diện sạch, dễ dùng trong thực tế. Em có thể thêm `clear` nếu muốn UX tốt hơn.

12) “Tại sao không xử lý `Ctrl+C` (SIGINT) để thoát nhẹ nhàng?”

Trả lời mẫu:

Hiện script không bắt tín hiệu. Nếu muốn xử lý ngắt để dọn dẹp, có thể thêm `trap`:

```
trap 'echo "Thoát..."; exit 0' INT TERM
```

Điều này giúp thoát gọn khi người dùng bấm `Ctrl+C`.

13) “Tại sao không dùng `select` để làm menu? Có nên đổi không?”

Trả lời mẫu:

`select` đơn giản để viết menu sẵn có và tự động đánh số lựa chọn, phù hợp cho scripts tương tác. `while true + case` cho phép kiểm soát linh hoạt hơn (ví dụ chấp nhận `c/C/1`). Dùng cái nào tùy mục tiêu; `select` tốt cho demo ngắn.

Kết luận (gợi ý nói với cô)

“Em sử dụng các lệnh này vì mục đích rõ ràng, đơn giản và dễ hiểu cho bài tập. Nếu cần production-ready, em sẽ: quote mọi biến, thay `ls|grep|wc` bằng `find`, dùng `read -r`, kiểm tra mã thoát sau `mv`, đổi `exit 1` thành `exit 0` khi thoát bình thường, và có thể thêm `trap` để xử lý `Ctrl+C`. Em sẽ cập nhật ngay nếu cô muốn phiên bản an toàn hơn.”

Muốn em **chuẩn hóa script luôn** theo các best-practices (ví dụ sửa phần đếm bằng `find`, thêm `read -r`, check lỗi `mv`, `exit 0`, và `trap`), mình sẽ gửi phiên bản sửa gọn trong nháy mắt. Em chọn sửa luôn chứ ạ?

Bài 5 – Vẽ tam giác bằng ký tự

bai5_LaMinhKhanh_4004267.sh

```
#!/bin/bash

draw_triangle() {
```

```

local char=$1
local size=$2

if ! [[ "$size" =~ ^[1-9][0-9]*$ ]]; then
    echo "Loi: Kich thuoc phai la mot so nguyen duong."
    return
fi

for ((i=1; i<=size; i++)); do
    for ((s=i; s<size; s++)); do
        echo -n " "
    done

    for ((k=1; k<=2*i; k++)); do
        echo -n "$char"
    done
    echo
done

}

char=$1
param=$2

if [ -z "$char" ] || [ -z "$param" ]; then
    read -p "Nhap vao ky tu hien thi: " char
    read -p "Nhap vao kich thuoc: " param
fi

draw_triangle "$char" "$param"

```

Dưới đây là cách em **giải thích rõ ràng, ngắn gọn** từng dòng để trả lời cô: “*Bài này em có hướng làm như nào*”. Em có thể đọc thẳng phần **Tóm tắt** khi thuyết trình, rồi bỏ sung phần **Giải thích chi tiết** nếu cô yêu cầu.

Tóm tắt (1–2 câu, để nói với cô)

"Em viết một hàm `draw_triangle` nhận 2 tham số: ký tự hiển thị và kích thước (số dòng). Hàm kiểm tra `size` là số nguyên dương, sau đó dùng 2 vòng lặp lồng nhau: một vòng in khoảng trắng để căn giữa, vòng kia in ký tự để tạo thành từng hàng của tam giác. Script đọc tham số từ dòng lệnh (nếu thiếu thì hỏi người dùng) và gọi hàm để vẽ."

Giải thích chi tiết (dòng-theo-dòng)

`#!/bin/bash`

- Shebang: khai báo shell dùng để chạy script (bash).

```

draw_triangle() {
    local char=$1
    local size=$2
    ...
}

```

- Định nghĩa hàm `draw_triangle`.
- `local char=$1` và `local size=$2` — gán tham số hàm vào biến cục bộ `char` và `size` (cục bộ trong hàm, không ảnh hưởng biến bên ngoài).

```
if ! [[ "$size" =~ ^[1-9][0-9]*$ ]]; then
    echo "Lỗi: Kích thước phải là một số nguyên dương."
    return
fi
```

- Kiểm tra tính hợp lệ của `size` bằng **biểu thức chính quy** trong `[[ ... ]]`.
 - `^[1-9][0-9]*$` nghĩa là một số nguyên dương (không cho 0, không dẫn 0).
 - `!` phủ định — nếu `size` **không** match thì in lỗi và `return` (kết thúc hàm, script tiếp tục chạy).
- **Ghi chú:** `return` chỉ thoát hàm; nếu muốn dừng toàn bộ script có thể dùng `exit 1`.

```
for ((i=1; i<=size; i++)); do
    for ((s=i; s<size; s++)); do
        echo -n " "
    done

    for ((k=1; k<=2*i; k++)); do
        echo -n "$char"
    done
    echo
done
```

- `for ((i=1; i<=size; i++))` — vòng ngoài lặp theo từng dòng của tam giác (từ 1 đến `size`).
- `for ((s=i; s<size; s++))` — in khoảng trắng trước phần ký tự để **căn giữa**; số khoảng trắng là `size - i`.
 - `echo -n " ":` in một khoảng trắng **không xuống dòng** (`-n`).
- `for ((k=1; k<=2*i; k++))` — in ký tự `char` lặp `2*i` lần trên dòng đó. (mỗi hàng in nhiều ký tự tạo phần “độ rộng” của tam giác).
 - `echo -n "$char":` in ký tự (không xuống dòng).
- `echo` sau hai vòng để **xuống dòng** (bắt đầu hàng mới cho `i+1`).

Lưu ý về kết quả: với `2*i` ký tự thì số ký tự từng hàng là 2,4,6,... (even). Nếu em muốn tam giác có số ký tự là 1,3,5,... (đỉnh 1 ký tự), nên dùng `2*i-1` thay vì `2*i`.

```
char=$1
param=$2

if [ -z "$char" ] || [ -z "$param" ]; then
    read -p "Nhập vào ký tự hiển thị: " char
    read -p "Nhập vào kích thước: " param
fi

draw_triangle "$char" "$param"
```


- `char=$1` và `param=$2`: lấy 2 tham số từ dòng lệnh (nếu chạy như `./bai5.sh * 5`).
- `if [ -z "$char" ] || [ -z "$param" ]`: nếu **bất kỳ** tham số nào rỗng thì hỏi người dùng bằng `read -p`.
 - `-z` kiểm tra chuỗi rỗng.
- `draw_triangle "$char" "$param"`: gọi hàm để vẽ tam giác với tham số đã có. Dấu ngoặc kép bảo đảm giá trị có khoảng trắng vẫn an toàn.

Một số điểm cần lưu ý / gợi ý cải tiến (để nói thêm với cô)

1. Số ký tự mỗi hàng:

- Hiện tại dùng `2*i` (2,4,6,...). Nếu em muốn tam giác có đỉnh 1 ký tự và tăng 2 ký tự mỗi hàng (1,3,5,...) thì dùng `for ((k=1; k<=2*i-1; k++))`.

2. Giới hạn ký tự hiển thị: nếu user nhập nhiều ký tự (ví dụ "ab"), script sẽ in chuỗi đó từng lần — có thể chỉ lấy ký tự đầu bằng `${char:0:1}` nếu muốn một ký tự duy nhất.

3. In khoảng trắng nhanh hơn: thay vì vòng lặp để in từng khoảng trắng, có thể dùng `printf`:

- `printf '%*s' $((size-i)) ""` để in `(size-i)` khoảng trắng.

4. Kiểm tra lỗi: hiện `return` chỉ kết thúc hàm; nếu `size` không hợp lệ mà em muốn dừng script luôn thì dùng `exit 1`.

5. Kiểm tra số lượng tham số: có thể kiểm tra và báo rõ ràng nếu `param` không phải số: `if ! [[ "$param" =~ ^[0-9]+$ ]]; then ...`

6. Unicode / multi-byte: nếu dùng ký tự UTF-8 rộng (ví dụ ký tự non-ASCII), việc căn chỉnh bằng số ký tự có thể phức tạp trên terminal — nhưng với ký tự đơn thường ổn.

Phiên bản script cải tiến (an toàn + đỉnh 1 ký tự + dùng `printf`)

Em có thể nộp phiên bản này nếu cần một phiên bản "sạch" hơn:

```
#!/bin/bash
```

```
draw_triangle() {
    local char=${1:0:1}          # chỉ lấy ký tự đầu nếu người dùng nhập nhiều ký tự
    local size=$2

    if ! [[ "$size" =~ ^[1-9][0-9]*$ ]]; then
        echo "Lỗi: Kích thước phải là một số nguyên dương." >&2
        return 1
    fi

    for ((i=1; i<=size; i++)); do
        # in (size-i) spaces bằng printf
        printf '%*s' $((size-i)) ""
        # in (2*i-1) ký tự để tạo tam giác với đỉnh 1 ký tự
        for ((k=1; k<=2*i-1; k++)); do
            printf '%s' "$char"
        done
    done
}
```

```

        done
        printf '\n'
    done
}

char=$1
param=$2

if [ -z "$char" ] || [ -z "$param" ]; then
    read -r -p "Nhập vào ký tự hiển thị: " char
    read -r -p "Nhập vào kích thước: " param
fi

draw_triangle "$char" "$param"

```

Ví dụ minh họa (để nói thêm nếu cô muốn kết quả mẫu)

- Nếu em chạy `./bai5.sh '*' 4` (hoặc nhập `*` và `4` khi script hỏi), tam giác in ra trông như:

```

*
***
*****
*****

```

Nếu cô hỏi sâu hơn, em có thể đọc từng dòng giải thích chi tiết ở trên, hoặc đưa ra phiên bản đã sửa (ví dụ tôi vừa cung cấp) để nộp. Muốn em chuyển luôn thành file hoàn chỉnh để nộp không?

Mô phỏng — câu hỏi cô có thể hỏi & cách trả lời ngắn gọn (kèm lý do và phương án thay thế)

Dưới đây là những câu hỏi thường gặp khi cô hỏi “*Tại sao dùng lệnh này mà không dùng lệnh khác?*” cho `bai5_LaMinhKhanh_4004267.sh`, kèm câu trả lời mẫu để em đọc nhanh lúc trình bày. Nếu cô muốn chi tiết hơn em đọc phần “Giải thích mở rộng”.

1) “Tại sao đầu file có `#!/bin/bash`?”

Trả lời mẫu: Đó là shebang — chỉ định script phải chạy bằng `bash` (vì em dùng cú pháp như `[[ ... ]]`, `for ((...))` chỉ có trong `bash`).

Khi đổi: Nếu muốn tương thích POSIX, tránh các cú pháp chỉ có trong `bash` và dùng `#!/bin/sh`.

2) “Tại sao em dùng hàm `draw_triangle()` thay vì viết lệnh trực tiếp?”

Trả lời mẫu: Dùng hàm giúp tách logic, dễ tái sử dụng và dễ test (gọi lại với các tham số khác). Khi code dài, hàm làm script rõ ràng hơn.

Khi đổi: Với script rất ngắn, viết thẳng vẫn được; nhưng hàm là best-practice.

3) “Tại sao khai báo `local char=$1` thay vì `char=$1`?”

Trả lời mẫu: `local` làm biến chỉ tồn tại trong hàm, tránh thay đổi biến toàn cục ngoài hàm — an toàn hơn khi script lớn.

Khi đổi: Nếu cần biến toàn cục thì bỏ `local`, nhưng thường không nên.

4) “Tại sao dùng `if ! [[ "$size" =~ ^[1-9][0-9]*$ ]]` để kiểm tra số nguyên dương?”

Trả lời mẫu: Dùng regex trong `[[ ... ]]` là cách trực tiếp để kiểm tra định dạng (loại trừ 0, cho phép nhiều chữ số). `[[ ... ]]` hỗ trợ regex trong `bash`.

Khi đổi: Có thể dùng `case "$size" in ''|*[!0-9]*) ...;; *) ...;; esac` hoặc `[[ "$size" =~ ^[0-9]+$ ]]` nếu cho phép 0. Nếu muốn kiểu portable POSIX, dùng `expr` hay `grep`.

5) “Tại sao dùng `return` khi size không hợp lệ, không `exit`?”

Trả lời mẫu: `return` chỉ thoát hàm, cho phép script tiếp tục xử lý (ví dụ có thể hỏi lại input). `exit` sẽ kết thúc toàn bộ script. Em chọn `return` để không tự vội dừng script.

Khi đổi: Nếu kích thước sai và muốn dừng hẳn, dùng `exit 1`.

6) “Tại sao in khoảng trắng bằng vòng lặp `for ((s=i; s<size; s++))` thay vì `printf`?”

Trả lời mẫu: Vòng lặp rõ ràng và dễ hiểu với người mới.

Khi đổi (khuyến nghị): `printf '%*s' $((size-i)) ""` in nhanh và hiệu quả hơn — tránh lặp nhiều lần và đặc biệt tốt khi `size` lớn.

7) “Tại sao in ký tự bằng `echo -n "$char"` thay vì `printf '%s' "$char"`?”

Trả lời mẫu: `echo -n` đơn giản, làm việc trên phần lớn hệ thống.

Khi đổi (khuyến nghị): `printf` có hành vi nhất quán hơn giữa các shell, an toàn với các ký tự đặc biệt, nên dùng `printf '%s' "$char"` cho production.

8) “Tại sao số ký tự mỗi hàng là `2*i` chứ không phải `2*i-1` (đỉnh 1 ký tự)?”

Trả lời mẫu: Em viết `2*i` nên đỉnh tam giác sẽ có 2 ký tự đầu. Nếu muốn tam giác với đỉnh 1 ký tự (thường gặp), phải dùng `2*i-1`. Em có thể sửa tùy yêu cầu.

Khi đổi: Thông thường dùng `2*i-1` để tam giác cân “truyền thống”.

9) “Tại sao không kiểm tra/chuẩn hóa char (ví dụ chỉ lấy 1 ký tự nếu user nhập nhiều ký tự)?”

Trả lời mẫu: Hiện em cho phép chuỗi nhiều ký tự — script sẽ in chuỗi đó lặp lại. Nếu cần chỉ một ký tự, có thể cắt `char=${char:0:1}` trước khi vẽ.

Khi đổi: Nên thêm ràng buộc nếu bài yêu cầu ký tự đơn.

10) “Tại sao dùng `read -p` mà không dùng `read -r -p`?”

Trả lời mẫu: `read -r` ngăn backslash là escape — an toàn hơn với input chứa `\`. Em nên dùng `read -r -p` để tốt hơn. Hiện `read -p` vẫn hoạt động nếu không có backslash trong input.

11) “Khi dùng ký tự Unicode (ví dụ tiếng Việt), script có căn chỉnh đúng không?”

Trả lời mẫu: Terminal và `echo/printf` với ký tự multi-byte có thể khiến căn giữa hơi lệch vì độ rộng ký tự khác nhau; `${#var}` cũng đếm byte/character tùy locale. Để chắc, cần test trên terminal UTF-8 hoặc dùng công cụ xử lý Unicode.

Khi đổi: Với ký tự wide/unicode có thể cần thư viện/ngoại công cụ để tính column width chính xác (không đơn giản bằng `bash` thuần túy).

12) “Tại sao không `validate param` khi đọc từ `read` (ví dụ bắt buộc nhập lại nếu không phải số)?”

Trả lời mẫu: Em có kiểm tra trong hàm; nhưng có thể thêm vòng `while` trước khi gọi `draw_triangle` để bắt nhập lại cho tới khi hợp lệ — tiện cho UX. Hiện script chờ cho hàm xử lý và `return` nếu không hợp lệ.

13) “Tại sao không in thông báo lỗi vào `stderr`?”

Trả lời mẫu: In ra console dùng `echo` là đơn giản. Tuy nhiên best-practice là in lỗi bằng `echo "... ">&2` để phân biệt `stdout` và `stderr`, nhất là khi script được pipe/redirect.

14) “Có cách nào làm nhanh/hiệu quả hơn khi size rất lớn?”

Trả lời mẫu: Dùng `printf '%*s'` cho khoảng trắng và `printf '%*s'/head` hoặc tạo một chuỗi dài sẵn rồi cắt — tránh vòng lặp nhiều lần in từng ký tự. `printf` và thao tác chuỗi sẽ nhanh hơn `echo` vòng lặp.

Mẫu câu tóm tắt để trả lời cô (1–2 câu)

“Em dùng hàm để tách logic; kiểm tra `size` bằng regex để đảm bảo là số nguyên dương; dùng vòng lặp in khoảng trắng và ký tự để căn giữa và xây hàng tam giác. Nếu cô muốn, em có thể đổi sang `printf` cho hiệu năng và dùng `2*i-1` để định chỉ 1 ký tự.”

Nếu cô hỏi sâu chỗ nào, em đọc tiếp phần giải thích tương ứng ở trên. Muốn em **gửi luôn phiên bản script đã chỉnh** (dùng `printf`, `2*i-1`, `read -r`, `validate` lặp lại, trim `char` thành 1 ký tự, in lỗi ra `stderr`) để em nộp luôn không?